Module 12: Manage Task Execution

DURATION: 3 HOURS | THEORY: 30% | HANDS-ON: 70%

This module focuses on **how automation actually runs under the hood**. Understanding task execution is critical for **scaling automation to hundreds or thousands of servers**.

Understand Execution

How Ansible executes tasks internally

Control Speed

Manage execution speed and parallelism

Optimize Performance

Tune playbook performance at scale

Troubleshoot

Diagnose execution and dependency issues

12.1 Understanding Task Execution

Before optimizing automation, it's important to understand **how Ansible executes tasks**. This foundational knowledge shapes every decision you make when writing and scaling playbooks across your infrastructure.

How Ansible Executes Tasks

Ansible follows a **task-based execution model**. When a playbook runs, it: loads the inventory, targets hosts, connects to each host, and executes tasks sequentially.

Example Playbook

```
- hosts: webservers
  tasks:
    - name: Install nginx
      package:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

Execution Flow

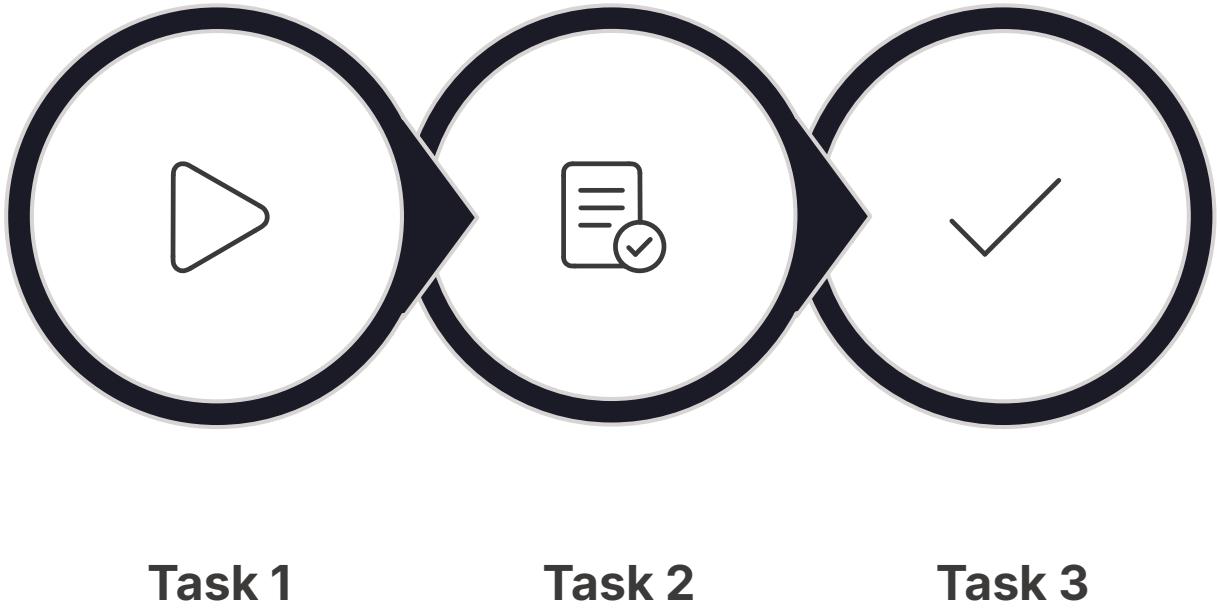
Ansible does **NOT** complete all tasks on one host first. Instead, it uses the **task-level execution model**:

```
Task 1 -> all hosts
Task 2 -> all hosts
```

Each task runs across **all hosts** before the next task begins.

Execution Sequence

With 3 hosts, tasks are distributed consistently across the entire fleet before advancing to the next step.



This model ensures a **consistent system state across infrastructure** — every host is always at the same task stage.

Enterprise Example

Consider a playbook updating a fleet of enterprise servers. The execution follows strict task-level ordering:

1

Step 1

Install patch on **all servers**

2

Step 2

Restart service on **all servers**

3

Step 3

Verify status on **all servers**

All servers complete **step 1 first**, then **step 2**, ensuring uniform state throughout the update cycle.

Lab Exercise

Students run a playbook and observe the task-level execution model in action. Notice how each server reports its status for the same task before the next task begins.

```
TASK [Install nginx]
ok: server1
ok: server2
ok: server3
```

Observe that all three servers complete the **Install nginx** task together before Ansible proceeds to the next task in the playbook.

Forks: Parallel Execution Explained

By default, Ansible runs tasks on **hosts in parallel**. The number of simultaneous parallel connections is called **forks**.

Default Setting

```
forks = 5
```

This means **5 servers are configured simultaneously** at any given moment during playbook execution.

Why Forks Matter

Forks directly control how fast your automation completes across large inventories. Too few forks = slow execution. Too many = controller resource exhaustion.

Tuning forks is one of the most impactful performance levers in Ansible.

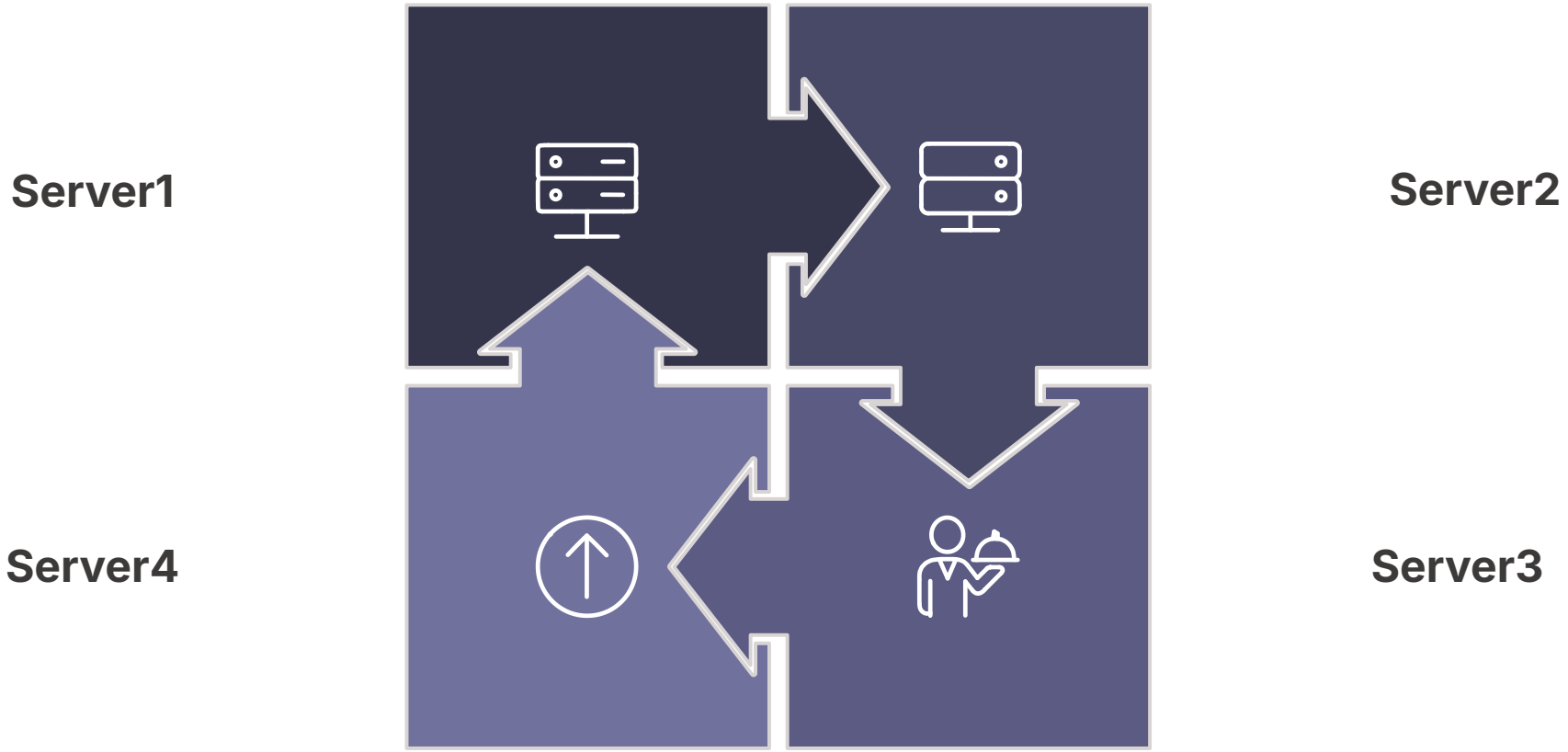
Forks in Action: Batching Example

With an inventory of **20 servers** and the default **forks = 5**, Ansible processes the fleet in four sequential batches:

Batch 1 Servers 1–5	**Batch 2** Servers 6–10
Batch 3 Servers 11–15	**Batch 4** Servers 16–20

Each batch of 5 servers runs tasks in parallel. The next batch begins only after the current batch completes.

Execution Diagram: Forks = 5



With **forks = 5**, five servers execute tasks simultaneously. Once that batch completes, the next group of five begins — continuing until all hosts in the inventory are processed.

Increasing Forks for Large Environments

Configuration

```
# ansible.cfg
[defaults]
forks = 50
```

Set in `ansible.cfg` or passed via CLI with `-f 50`.

When to Increase Forks

Large environments benefit significantly from higher fork values. Increasing forks reduces total execution time by processing more hosts simultaneously.

- Used for **hundreds of servers**
- Requires sufficient controller CPU and memory
- Network bandwidth must support parallel SSH connections

Enterprise Example: 500 Servers

500

Linux Servers

Total fleet size in this enterprise environment

100

Fork Setting

Parallel connections configured for this scale

5x

Speed Gain

Automation finishes **much faster** vs. default forks

Connection Types

Ansible communicates with hosts using **connection plugins**. Different platforms require different connection methods — choosing the right one is essential for reliable automation across heterogeneous environments.

SSH: Linux Systems

Configuration

```
connection: ssh
```

SSH is the **default connection** for Linux and Unix-based systems. No additional setup required on modern Linux hosts.

```
ansible web -m ping
```

Used For

- Linux servers
- Unix systems
- Network devices

SSH provides encrypted, authenticated communication and is the most widely used connection plugin in Ansible deployments.

WinRM: Windows Systems

Configuration

```
connection: winrm
```

Windows hosts use the **WinRM protocol** (Windows Remote Management) instead of SSH. Requires WinRM to be enabled and configured on target hosts.

```
- hosts: windows
  connection: winrm
```

Required For

- Windows Server
- Active Directory
- IIS servers

The `pywinrm` Python library must be installed on the Ansible controller to use WinRM connections.

Local Connection

Used when tasks run **on the controller itself** — no remote connection is made. Ideal for automation that interacts with APIs or cloud platforms rather than remote hosts.

```
- hosts: localhost
  connection: local
```

Cloud Provisioning

Spin up cloud resources via API calls executed locally on the controller.

API Automation

Interact with REST APIs, webhooks, and external services without SSH.

CI/CD Scripts

Run pipeline tasks, tests, and deployment scripts directly on the controller.

Enterprise Example: Hybrid Environment

Real-world enterprise environments are rarely homogeneous. Ansible handles mixed infrastructure by applying the correct connection plugin per host group:

Linux Hosts

SSH — default, encrypted, agent-based authentication

Windows Hosts

WinRM — Windows Remote Management protocol

Cloud APIs

Local — controller executes API calls directly

Understanding Task Output

When tasks run, Ansible produces **structured output** for every task on every host. This output is your primary window into what automation is doing — and what it changed.

```
TASK [Install nginx]
ok:      server1
changed: server2
failed:  server3
```

Each line tells you the **host name** and the **result status** of that task — making it easy to identify which servers need attention at a glance.

Output Status Types

Status	Meaning
`ok`	Task already in desired state — no changes made
`changed`	Task successfully modified the system
`failed`	Task encountered an error and failed
`skipped`	Task skipped due to a `when` condition
`unreachable`	Host connection failed — could not reach target

Reading Output: Example

Task Output

```
changed: server1
ok:      server2
```

What It Means

- **server1** — package was **not installed**; Ansible installed it (`changed`)
- **server2** — package was **already installed**; no action taken (`ok`)

This distinction is the foundation of Ansible's idempotent behavior — the system only changes what needs changing.

Why This Matters: Idempotency

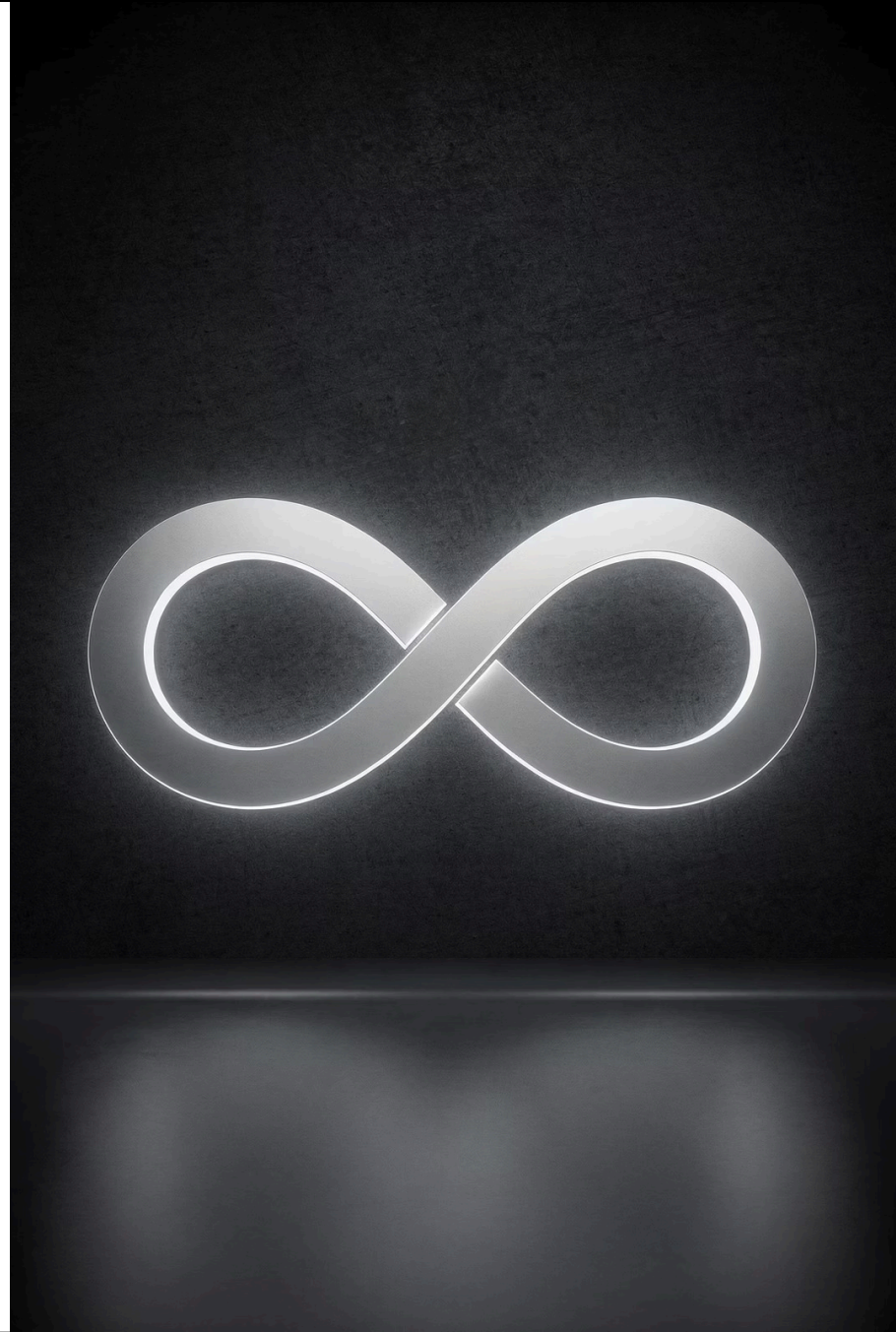
> Ansible is **idempotent** — repeated runs do **not break systems**.

Safe to Re-Run

Running the same playbook multiple times produces the same result. Systems already in the desired state are left untouched (`ok`).

Drift Detection

A `changed` status on a re-run signals configuration drift — something changed outside of Ansible and was corrected.

Confidence at Scale

Idempotency means you can safely automate hundreds of servers without fear of unintended side effects from repeated execution.